

DNAttest: Digital-twin-based Non-intrusive Attestation under Transient Uncertainty

Wei Lin*, Heng Chuan Tan[†], Binbin Chen*, Fan Zhang[‡]

*Singapore University of Technology and Design, Singapore

[†]Advanced Digital Sciences Center, Singapore

[‡]Zhejiang University, Hangzhou, China

Email: wei_lin@mymail.sutd.edu.sg, hc.tan@adsc-create.edu.sg, binbin_chen@sutd.edu.sg, fanzhang@zju.edu.cn

Abstract—Programmable logic controllers (PLCs) are vulnerable to malware, which is a key security risk for Industrial Control Systems (ICSs). Existing attestation solutions are invasive because they require hardware security modules and software upgrades in legacy devices. We propose DNAttest, a Digital-twin-based Non-invasive Attestation solution to attest PLC behaviors in near-real time. DNAttest requires minimal ICS infrastructure changes and does not interfere with normal ICS operations. DNAttest detects PLC deviations by replicating all input messages for a PLC to its digital twin and comparing their output messages. Due to transient uncertainty in the PLC's internal processing state, DNAttest may output an incorrect comparison. To generate all plausible output values for comparison, we instantiate multiple emulated PLCs by replicating input messages with different timing profiles. We demonstrate on a close-to-real-world power grid testbed that DNAttest can provide a timely detection of a wide range of attacks non-invasively and accurately. DNAttest solution is lightweight and scalable. A typical desktop PC can attest more than 20 actual PLCs even if we use 10 emulators to monitor every actual PLC.

Index Terms—Industrial control systems, programmable logic controllers, attestation, code integrity checking

I. INTRODUCTION

Programmable Logic Controllers (PLCs) are deployed in Industrial Control Systems (ICSs) to control critical infrastructures (e.g., energy, water and transportation) [1]. They are increasingly being targeted by attackers to disrupt industrial processes [2]–[5]. Such attacks are possible because of the lack of authentication and unintended exposure to the outside world [6]. These vulnerabilities allow attackers to design stealthy malware (e.g., Harvey rootkits [7] and ladder logic bombs [8]) to alter control logic and PLC firmware [9]. Some PLC models also contain inherent flaws [10], [11] that allow attackers to modify control logic undetected.

One method for dealing with PLC attacks is to perform hardware attestation using a secure hardware root-of-trust module. However, retrofitting hardware modules may be too costly for legacy PLCs. Another method is software attestation, which requires the PLC to generate a correct checksum for its memory content within a specific time. Unfortunately, it requires an upgrade of the PLC software, which involves vendor support and costly recertification. Furthermore, hardware and software attestation are intrusive and vulnerable to time-of-check-to-time-of-use (TOCTTOU) attack [12].

In this work, we propose a Digital-twin-based Non-intrusive Attestation (DNAttest) to verify the PLC's runtime behaviors. Unlike conventional remote attestation approaches, we assume that existing PLCs lack hardware support and privileged access to upgrade their software. Instead, DNAttest is based on emulating a real PLC's behavior in a high-fidelity digital twin to perform near real-time attestation. In other words, DNAttest runs on a separate PC alongside the actual PLC. It does not interfere with ICS operations and operates by replicating all input messages from the real PLC to the emulated PLC. Given the same input network messages, the emulated PLC running an identical uncompromised copy of the control logic¹ should output the same outgoing messages as the real PLC. A comparator module then passively compares the real and emulated PLC output messages. If there is a discrepancy in their output, it indicates an attack.

While the basic intuition is simple, its design and implementation are non-trivial due to the lack of visibility into the real PLC's internal state. Consider a scenario where a real PLC receives an input message and periodically generates an output message. If the input message arrives too close to the time of generating the output message, then there is uncertainty whether the value of the output message reflects the input message that just arrived. In this case, DNAttest will generate a false alarm if the internal processing state of the input message is different between the real PLC and the emulator PLC. However, this uncertainty only lasts for a short period of time. The real PLC will eventually process the input message and update its output message accordingly. Hence, we call such behavior “transient uncertainty”. We note that transient uncertainty affects not only the real PLC but also the emulated PLC.

To address transient uncertainty, DNAttest instantiates multiple PLC emulators and varies the arrival time of replicated messages to enumerate all possible transient scenarios. Because the uncertainty is transient, we only need a small number (e.g., we show that 10 is sufficient) of emulator instances to cover the different scenarios even under the most intensive traffic we observed in an electric ICS. Furthermore, we can reuse the same set of emulated PLCs over time.

¹The plant operator must reupload the control logic to the emulated PLC when the PLC logic is updated. The operator must verify that the updated code is accurate and properly authenticated.

By coordinating multiple emulator instances, our experiments show that DNAttest can provide accurate and near real-time detection of a wide range of attacks on a commodity PC. Specifically, our contributions are as follows:

- We design and implement DNAttest, a practical non-intrusive attestation solution that overcomes various real-world constraints faced by existing attestation solutions. We automate the construction of DNAttest based on popular PLC control logic software. Our prototype also supports multiple ICS protocols.
- We identify the problem of transient uncertainty which is fundamental to the design of such a non-invasive solution and propose a novel design that coordinates multiple emulator instances to provide accurate and timely detection under such transient uncertainty.
- We show through extensive experiments that DNAttest can keep up with the real PLC when deployed in tandem on a commodity PC. We apply DNAttest to the Electric Power and Intelligent Control (EPIC) testbed [13], [14], a high-fidelity power grid testbed with simulated attack scenarios and show that DNAttest can detect various attacks in near real-time without false alarms. Our experiments also validate that the DNAttest solution is lightweight and scalable — a single PLC emulator uses less than 4% of the computational power of a single Intel i7-10700 core and less than 70 MB of memory — even if we use 10 emulators to monitor every actual PLC, a typical industrial PC with an 8-core Intel i7-10700 CPU and 16 GB of memory (the whole setup costs less than 2000 USD) can attest more than $8/(0.04 \times 10) = 20$ actual PLCs — which are sufficient to attest all the PLCs in a medium-size ICS that costs hundreds of thousands of USD or more. Hence, the relative hardware and computing cost by DNAttest will be only 1% or less as compared to the cost of the ICS being attested.

The rest of the paper is organized as follows. Section II reviews the state-of-art attestation techniques. Section III presents the threat model and system assumptions. In Section IV, we explain the basic design, where we use only one emulator to validate the real PLC's output. In section V, we explain the transient uncertainty problem and explain how it can be avoided by using multiple emulators. Section VI presents a case study related to the synchronization process of the EPIC testbed followed by system implementation of DNAttest in Section VII. In Section VIII, we conduct several experiments to validate the effectiveness of our proposed framework. Finally, we conclude our paper in Section IX.

II. RELATED WORK

Hardware-based Attestation relies on a dedicated hardware root-of-trust to verify device software [15]. One example is Trusted Platform Module (TPM), a secure cryptoprocessor designed to self-destruct if physically compromised. It verifies software integrity via encrypting, hashing, and signing operations. Trusted hardware also includes Trusted Execution Environment (TEE), such as ARM's TrustZone [16] and Intel's

Software Guard Extension (SGX) [17], which are becoming the de facto standard for commercial-off-the-shelf (COTS) devices [18]. Unlike TPMs, TEE is integrated by manufacturers into chipsets. Thus, it provides secure storage and isolated execution for greater security. However, these technologies require hardware modifications. In addition, they have a high computational overhead, making them unsuitable for delay-intolerant applications and resource-constrained ICS devices.

Software-only Attestation uses trusted software to verify software integrity [19]. This technique does not require hardware support (e.g., see [20], [21]). Instead, it uses the following elegant idea: If the running copy of some carefully designed checksum software code is unmodified, it should be able to compute a checksum over its own memory footprint faster than any modified versions. Intuitively, the checksum (seeded with some random nonce) represents a fingerprint of the authentic executable image, and any modified software must do extra work to "hide" its different memory footprint. While software-only attestation has a lower adoption bar for legacy devices, it has strict requirements and security weaknesses [22]. In particular, no untrusted proxy device should help the prover generate the checksum. In addition, some ICS devices require extensive testing before software modifications can be made, making it difficult to add attestation support.

Behavior-based Attestation builds a predictive model (based on the physical properties of the system) and compare the model with actual system behavior to detect attacks. It is applicable to existing PLCs without special hardware or direct memory access. However, building an accurate model that precisely characterizes the behavior of the system is difficult. Several approaches have been proposed to automatically construct such models. In [23], the authors proposed a machine learning-based approach to construct invariant conditions from ICS sensor data. This approach has been successfully demonstrated on a realistic water testbed. However, this approach requires mutation of the PLC code (i.e., to mimic an attacker) and collection of 'anomalous' data to train a classifier model, making it impractical for real-world critical infrastructure due to its high cost and security concerns. Instead of mutating the PLC code, the authors in [24] perform an information flow analysis to identify critical inputs from the PLC, then repeatedly mutate those input values to determine the output and build the training dataset. Although this method improves data collection, it assumes that the PLC code or at least translations of the actual PLC programs are available in suitable formats for input mutation. More recently, authors have used static program analysis in [25]–[27] to determine the causal relationships between events in the PLC code and extract temporal invariants (i.e., physical models) or intrusion detection logic that model the dependency between machine operations. However, their approach requires static analysis of PLC code, which may not be practical (since source code may not be available), and (2) may produce many false positives due to the limitations of static analysis.

State Machine Replication approach creates multiple application replicas that interact with client requests to implement

fault-tolerant services [28]. The replicas use voting to reach a consensus on how to process commands that may change the state. While the replication technique is similar to our work, the main objective for them is to tolerate fault and deal with the Byzantine problem [29] in distribution systems. In state machine replication, multiple replicas must coordinate with one another to ensure that a majority of non-faulty ones can provide intended services. DNAttest, on the other hand, replicates network inputs to the emulated PLCs (i.e., replicas) to attest the real PLC outputs. There is no interaction between the real and the emulated PLCs. The emulated PLCs are outside the critical paths of plant operation and their outputs are used to attest the behavior of real PLC instead of improving fault tolerance. Also, DNAttest need to deal with the practical challenge that the internal states of the real (and oftenly, the emulated) PLC is invisible.

III. THREAT MODEL AND SYSTEM ASSUMPTIONS

Our work focuses on the threat model that the PLC control logic is manipulated by an attacker. The attacker can alter the PLC runtime behavior in many ways. The first vulnerability assumes the attacker has administrative access to the PLC software (e.g., CoDeSys [30] and Rslogix 5000), allowing the attacker to intercept the compilation process and inject code into the control logic [11]. The second vulnerability exploits flaws in remote protocols to remotely communicate with the PLC, allowing the attacker to modify control logic or binary code without using the PLC software [10]. The third vulnerability is related to insecure ICS protocols, where an attacker can launch man-in-the-middle attacks to craft malicious network packets in real-time to manipulate the industrial process [14]. It is also possible to compromise the PLC control logic through physical access by connecting an infected USB drive to the PLC or via debugging interfaces such as JTAG [7], [8]. Through various means, an attacker can launch command injection attacks, including modifying payload, delaying, and deleting actuation commands.

We assume that the sensor inputs and commands from the human machine interface (HMI) and supervisory control and data acquisition (SCADA) are uncompromised. Their cybersecurity risk is mitigated using other complementary defense mechanisms, e.g., [31]–[35]. Hence, attacks to the PLC inputs are outside the scope of our work. We assume that DNAttest cannot address sensors and actuators that are hardwired to the PLC. Multiplexer devices must be used to replicate these hardwired signals to DNAttest in such contexts. However, many ICSs (e.g., IEC 61850-based substations) have eliminated hardwiring in favor of transmitting all data over Ethernet. As this transition continues, we believe that DNAttest will play an even greater role in the near future. In our design, DNAttest instantiates multiple PLC emulators to model the real PLC behaviors. While our experiments showed that this can easily be supported by running VMs on standard PCs with moderate configuration (see Table III), the setup cost increases as the system scales. For a large-scale deployment with hundreds of PLCs, we can deploy the emulated PLCs on

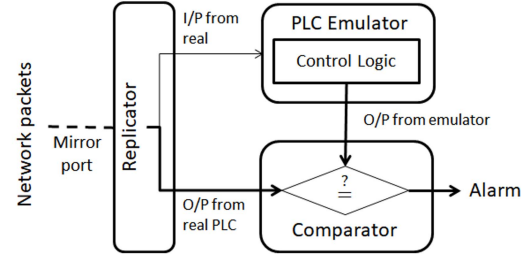


Fig. 1: The building blocks of DNAttest framework

a private cloud or edge infrastructure (hosted by ICS operators and consisting of COTS servers) by redirecting relevant traffic via data diodes to isolate the real ICS systems from external attacks. This migration allows operators to lower setup cost by leveraging mature IT solutions and economies of scale, making our solution more practical.

IV. DNATTEST : THE BASIC DESIGN

DNAttest runs on a commodity PC to receive traffic to and from the targeted (real) PLC. It does not send any traffic to the real ICS. This requirement is achieved by tapping the mirror port of the Ethernet switch to which the real PLC is connected. The central idea is to validate the real PLC behavior (i.e., output messages — their content and timing). Fig. 1 shows the DNAttest architecture comprising a replicator, an emulated PLC, and a comparator. The replicator VM passively mirrors the input messages from the real PLC and forwards them to the emulated PLC. The emulated PLC executes the same control logic as the real PLC. The comparator compares the emulated and real PLC output. If the real PLC is uncompromised, the emulated output should match the real PLC or else the comparator will alert the ICS operator of an attack.

A. PLC Operation Model

Before describing each component of our DNAttest framework, we first explain the PLC operation model to motivate our design. The PLC acts as an interface between the SCADA level and the field devices to provide real-time monitoring and control of the physical processes. Every PLC has a scan cycle that comprises an input scan to receive sensor inputs and commands from SCADA/HMI, a program execution phase that executes user-defined control logic, and an output generation phase that updates output values to actuate field devices.

As shown in Fig. 2, if an input message *in1* arrives before an input scan (in scenario 1), it will be processed by the PLC and it can trigger an *out1* in the output phase from the CPU of the PLC, which will subsequently be output to the network after some variable delay in the PLC's network stack. We denote an output triggered by an input message an event-triggered mode, while an output triggered by an internal timer in the PLC a timer-triggered mode. Note that in scenario 1, the *out1* can be triggered by either the event (arrival of *in1*, or the timer, or both). If the same input *in1* arrives after the input scan, it will not be processed by the PLC in time to produce an output in the same scan cycle. Hence, in scenario 2, *out1* is not caused

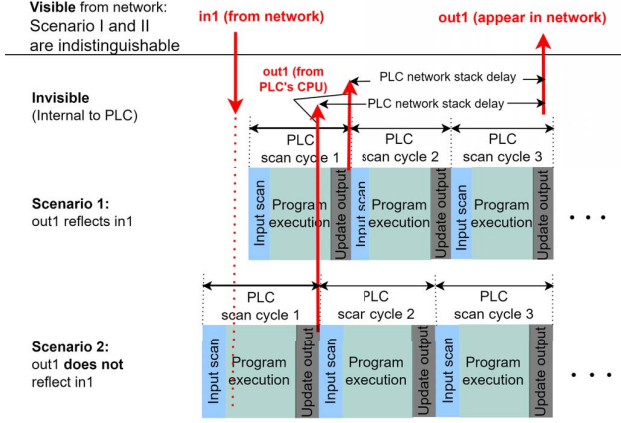


Fig. 2: An illustration of how a PLC processes an input.

by $in1$ but by a periodic timer, i.e., it is a timer-triggered output. We also assume that the PLC processes inputs in the same order as they arrive in the network. As such, our design ensures that the same inputs are processed by all the PLC emulators in the same order.

Our DNAttest design can support both event-triggered outputs and periodic timer-triggered outputs or a mix of both types of triggering. For ease of understanding in this paper, we focus on the case that output messages from PLC are triggered only by periodic timers, while allowing their values to depend on the input messages that arrive in an aperiodic manner.

While more description will be given later when we formally define transient uncertainty, Fig 2 shows that by just looking at the order and time difference between the input message $in1$ and output message $out1$ at the network, it is impossible to distinguish whether the $out1$ reflects $in1$ or not (i.e., the scenario 1 and 2 both can happen inside the PLC, which is a state invisible to an external observer).

B. Replicator

The replicator captures packets through the mirror port of the switch and stores them in a queue. Its function is to replicate stored packets to the emulated PLC. Since most ICS protocols use TCP/IP, forwarding packets directly to the emulated PLC will cause TCP errors. Thus, we instantiate temporary client/server emulators to establish a TCP handshake with the emulated PLC. Once the TCP connection is established, the replicator enters the packet forwarding state. In this state, the replicator duplicates network packets in the queue and modifies the (i) source and destination MAC addresses, (ii) IP identification field, source and destination IP addresses, (iii) TCP sequence and acknowledgment numbers, (iv) ICS protocol header fields, including (v) packet length and checksum to redirect packets to the PLC emulator.

Note that any compromised inputs (e.g., sensors and SCADA/HMI commands) that target the real PLC can reach the emulated PLC, causing both real and emulated PLCs to produce identical outputs and DNAttest to miss an attack. To deal with malicious inputs, we can implement sanity checks

in the replicator to discard them from being replicated to the emulated PLCs. To handle excessive message spamming that causes DNAttest to instantiate unnecessary emulated PLCs to deplete system resources, we can implement a rate limiter in the replicator to block incoming messages beyond a threshold.

C. Emulated PLC

We use CoDeSys runtime [30] to emulate the real PLC. The runtime toolkit supports various PLC programming languages defined in the IEC 61131-3 standard [36] and is compatible with over 330 device manufacturers. Using CoDeSys runtime, we can perform high-fidelity emulations using the same code and libraries as a real PLC. As shown in Fig. 1, the emulated PLC is connected to the replicator. It implements CoDeSys runtime to process input packets from the real PLC. Then the results are sent to a comparator for verification. To ensure the same functionality as the real PLC, the emulated PLC must establish a TCP connection with the client/server emulator, as previously described. To deal with input attacks that bypass our replicator checks, we can instrument another emulated PLC with additional runtime tools such as PurifyPlus [37], Memory-Validator [38], and binary hardening techniques (e.g., redzones and low-fat pointers) [39] to periodically check the emulated PLC memory for malicious inputs. Compared to verifying the real PLC's binary image at runtime, instrumenting an emulator requires minimal effort at a lower overhead without affecting plant operations or attestation capability.

D. Comparator — Basic Detection

The comparator implements a dictionary to store and compare the real and emulated PLC output messages, which are the payload of industrial protocols (e.g., Modbus [40], and IEC 61850 Manufacturing Message Specification — MMS [41]). Any mismatch in the payload means that the PLC is compromised². Detection is based on the following observations:

- The ICS payload sent by the real and emulated PLC must be identical; otherwise, it is a modification attack.
- For timer-triggered periodic outputs, the inter-arrival time between two outputs must not be greater than Π_m ³, else it is a delay attack. If a message is missing within Π_m , it is a deletion attack. If there are multiple packets of the same type within Π_m , it is an injection attack.

Basic Approach. We hash the ICS protocol payload and store the hash value in a dictionary for comparison later. Some ICS payloads may contain fields that are generated on-demand, e.g., timestamp values to indicate state change. We ignore such fields in the hash calculation. Next, we create a timer with appropriate offset to account for replication and packet processing delays. The dictionary then binds the timer to each packet and check if the real and emulated PLC output the same hash value within a certain delay. If a match is found, the

²We do not consider other protocol layer attacks such as ARP spoofing and TCP SYN because there are existing techniques to address them.

³ Π_m can be determined from PLC config. file or captured network traces.

Algorithm 1: Basic Detection Logic

```

1 Procedure comparator( $O$ )
2    $V \leftarrow h(O)$  //  $O \rightarrow$  emulated or real output
3   if  $V$  exists in  $Dict$  then
4     if  $V$  from different side then
5       matchedCounter ++
6       cancel timer(key= $V$ )
7       remove ( $V$ ) in  $Dict$ 
8     end
9   else
10    insert ( $V$ ) in  $Dict$ 
11    start timer(key= $V$ , handle_matching_timeout,
12      detection_delay,  $O$ )
13  end
14 Function handle_matching_timeout( $O$ ):
15   unmatchedCounter ++
16   if  $unmatchedCounter / matchedCounter > threshold$ 
17     then
18       raise alarm( $O$ )
19       reset matchedCounter
20       reset unmatchedCounter
21   end
22   remove ( $h(O)$ ) from  $Dict$ 

```

comparator will remove the hash value and the corresponding timer from the dictionary.

For an accurate comparison, the real and emulated PLC must generate exactly the same number of output messages within a specific delay. This is difficult to achieve because the emulated PLC runs on a different platform than the real PLC. One of the PLCs might operate at a faster processing speed than the other, resulting in a mismatch in the different number of output messages. Furthermore, the internal states of the real and emulated PLCs may not be synchronized due to transient uncertainty. Hence, basic detection (non-colored rows in Algorithm 1) will generate many false alarms.

Dealing with false alarms. We can suppress false alarms by introducing two counters to track the number of matched and unmatched output messages (see lines 5 and 14 in Algorithm 1). We then implement threshold checking (line 15) to raise alarms when the number of unmatched-to-matched ratio exceeds the predefined threshold. The threshold can be determined based on the difference in the processing speed of the real and emulated PLC. While this reduces false alarms, it also increases the chances that a true attack will go undetected. Algorithm 1 can be further improved by keeping an array of the most recent output message for each type. When there is no exact match in the current time interval, the comparator will try to find a match in the array from the previous interval. However, this approach cannot completely eliminate false negatives because an advanced attacker who knows the detection logic can replay a previous valid value in the current timestamp to bypass detection.

V. TRANSIENT UNCERTAINTY

In this section, we discuss the effects of transient uncertainty and present solutions to improve the basic approach (i.e.,

TABLE I: List of notations for explaining transient uncertainty.

Notation	Description
$in\{i\}$	The i th incoming message to PLC.
$out\{i\}$	The i th outgoing message from PLC.
$T_{in\{i\}}$	Arrival time of the $\{i\}$ th incoming message to PLC.
$T_{out\{i\}}$	Timestamp of the $\{i\}$ th outgoing message from PLC.
Δ_{Min}	Lower time bound of transient interval.
Δ_{Max}	Upper time bound of transient interval.
Π_m	Sending interval of a periodic output message type m .

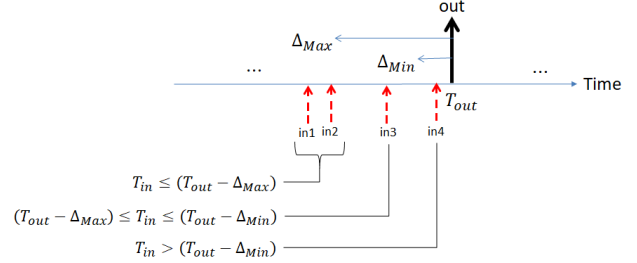


Fig. 3: Case 1: A single message in the transient interval.

Algorithm 1). Related notations for transient uncertainty are introduced in Table I.

A. A Fundamental Challenge for Non-invasive Attestation

Recall that transient uncertainty interval is the interval between an input message and an output message for which the value of the output message may not reflect the input message. This transient effect is illustrated in Fig. 2 where both scenarios 1 and 2 are possible and indistinguishable at the network based on the output packet time difference. To better illustrate transient interval, we define Δ_{Max} and Δ_{Min} as timing bounds for a PLC's transient interval in Fig. 3, which determines whether an input message is processed. Transient uncertainty occurs when the PLC receives more than one input messages between $T_{out} - \Delta_{Max}$ and $T_{out} - \Delta_{Min}$ for an output message at T_{out} . No transient uncertainty occurs for input messages that arrive before $T_{in} \leq T_{out} - \Delta_{Max}$ and later than $T_{out} - \Delta_{Min}$. Formally, Δ_{max} and Δ_{min} values depend on the variable duration of the PLC's CPU input scan (I_{min} to I_{max}), program execution (E_{min} to E_{max}), output generation (O_{min} to O_{max}), and the PLC's network stack delay (N_{min} to N_{max}). Consider an input message arrives before the end of the input scan phase, we can derive that: $\Delta_{min} = E_{min} + O_{min} + N_{min}$ is the minimum time gap between an input message and an output message that it could possibly change. On the other hand, $\Delta_{max} = I_{max} + E_{max} + O_{max} + N_{max}$ denotes the largest interval between an input message and an output message whose output value does not reflect that input message.

Suppose the outputs of $in1$, $in2$, $in3$, and $in4$ are $out1$, $out2$, $out3$, and $out4$, respectively. We can describe transient uncertainty and the values of the expected output messages from the PLC based on the following two cases.

- Case 1: A single message arrives in the transient interval. As shown in Fig. 3, $in4$ arrives at $T_{in} > T_{out} - \Delta_{Min}$, $in1$ and $in2$ before $T_{out} - \Delta_{max}$. Only $in3$ arrives

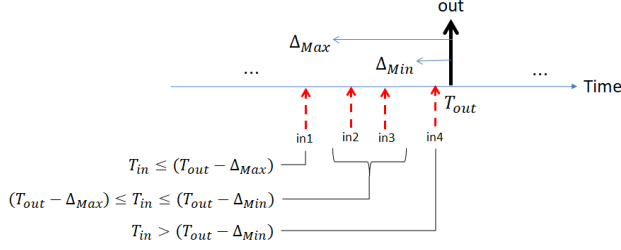


Fig. 4: Case 2: Two messages in the transient interval.

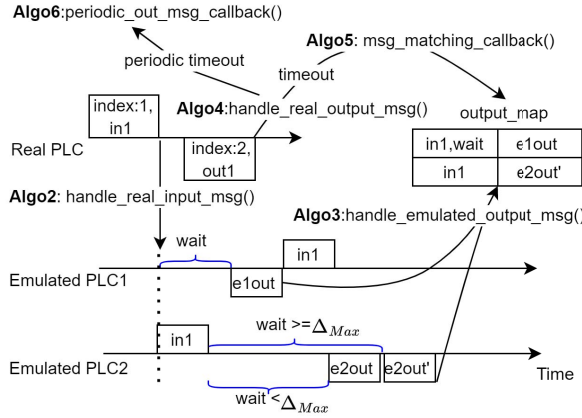


Fig. 5: An example to illustrate the working of main procedures (Alg. 2 to Alg. 6) in DNAttest.

between $T_{out} - \Delta_{Max}$ and $T_{out} - \Delta_{Min}$. In this case, the PLC output is either $out3$ (if $in3$ is processed) or $out2$ if $in1$ and $in2$ are processed. Note that $in4$ arrives too late to be processed.

- Case 2: two (or more) messages arrive in the transient interval. As shown in Fig. 4, $in2$ and $in3$ arrive between $T_{out} - \Delta_{Max}$ and $T_{out} - \Delta_{Min}$. In this case, there are three legitimate possibilities. If $in2$ and $in3$ are not processed, the output is $out1$. If $in2$ and $in3$ are processed, the PLC output is $out3$. And if $in2$ is processed but not $in3$, the output is $out2$. In all cases, $in4$ will not be processed by the PLC because it arrives too late.

If the comparator relies on a single outcome from the emulator PLC to compare multiple (> 1) possible outcomes from the real PLC, many false positives will arise.

B. Dealing with Transient Uncertainty (Single Input Message)

We solve the transient uncertainty problem by instantiating two PLC emulators to handle one input message. The rationale is to account for all possible output values of the real PLC when transient uncertainty occurs. In other detail, we design a set of functions (see Algorithms 2 to 6) that are triggered either by a message (at the real PLC or the emulated PLC) or by the expiration of some timers, as shown in Fig. 5. First, we consider the arrival of an input message $in1$ at the real PLC, which will trigger a periodic output message $out1$.

Algorithm 2: Handle the real PLC's input message

Input: $emulators$: array of emulators.
Input: I_{real} : real PLC's input message
Output: assign task to individual emulator

```

1 Function handle_real_input_msg( $I_{real}$ ):
2   available_count = check_emulator_availability()
3   while available_count <= 2 do
4     activate_new_emulator()
5     available_count = check_emulator_availability()
6   end
7   count = 0
8   for each emulator in emulators do
9     if emulator.status == AVAILABLE then
10      if count < 2 then
11        if count == 0 then
12          emulator.isWait = True
13        end
14        emulator.status = ASSIGNED
15      end
16      emulator.assign_input_task( $I_{real}$ )
17      count ++
18    end
19  end

```

Algorithm 3: Handle the output message from an emulated PLC in the ASSIGNED state

Input: O_{emul} : emulated PLC's output message

```

1 Function handle_emulated_output_msg( $O_{emul}$ ):
2    $I_{emul} \leftarrow InQueue_{emul}.get(emulator.current\_step)$ 
3    $t_{interval} = O_{emul}.timestamp - I_{emul}.timestamp$ 
4   if  $t_{interval} \geq \Delta_{max}$  then
5     output_map[emulator.current_step] =  $O_{emul}$ 
6     emulator.status = AVAILABLE
7     emulator.isWait = False
8     bring_to_latest(emulator.latest_step)
9   end

```

Algorithm 2. When the replicator detects an input message $in1$ from the real PLC, it invokes function `handle_real_input_msg()` in Algorithm 2 to dispatch message $in1$ to two emulated PLCs (we initialized 10 emulated PLCs at the beginning). For emulated PLC1, we wait for it to send an output message i.e., $e1out$ (which may be due to a previous input message or caused by a periodic timer) before sending the replicated input message $in1$. This wait accounts for the case when the real PLC generates $out1$ without processing $in1$. To handle the case where the real PLC has processed $in1$, we send message $in1$ to the emulated PLC2 without delay. However, $in1$ may or may not be processed due to the inherent transient uncertainty faced by the emulated PLC2 itself. Since we require emulated PLC2 to process $in1$, we must ensure that the output from the emulated PLC2 is at least Δ_{Max} later than the time we feed $in1$ to the emulated PLC2. If the output is less than Δ_{Max} (e.g., $e2out$ in Fig. 5), it is likely that its value is due to the result of a previous input or a periodic timer and thus must be discarded. As shown in Fig. 5, only output $e2out'$ is considered because it appears more than Δ_{Max} after input message $in1$. Once the emulated PLC1 and PLC2 have been assigned an input message, we change their status from AVAILABLE to ASSIGNED.

Algorithm 3. Next, we wait for the emulated PLC1 and

PLC2 to generate the required output. Algorithm 3 then invokes function `handle_emulated_output_msg()` to store the output values to the `output_map`. We calculate the time difference between the input message and the emulated output message. If the time difference is $\geq \Delta_{Max}$, we store the current step of the respective emulators as a key and their corresponding outputs O_{emul} as a value in the `output_map`. For emulated PLC1, we define the current step (i.e., the key in the `output_map`) as "`in{i},wait`" and emulated PLC2's current step as "`in{i}`" to differentiate the different handling of input message `in{i}`. After storing the emulated outputs, we bring forward emulated PLC1 to the latest state by sending input message `in{i}` to it (so that it is synchronized with emulated PLC2 to process future messages). Finally, we reset the status of emulated PLC1 and emulated PLC2 to AVAILABLE.

Algorithm 4 When the replicator detects an output `out1` from the real PLC, it retrieves from its input buffer `InQueuereal` a list of input messages that fall within the transient period (i.e., from $T_{out} - \Delta_{Max}$ to $T_{out} - \Delta_{Min}$ interval) by checking whether the time difference between each input message and the real PLC output is $\leq \Delta_{Max}$. Additionally, the replicator retrieves the most recent input message that lies outside $> \Delta_{Max}$ to account for the case where the output message is caused by an input message in the previous scan cycle. The replicator then populates those input messages into an INPUTCHECKLIST. Specifically, for each input message `in{i}`, the replicator stores two entries, i.e., (1) the index of the input message (e.g., "`in{i}`"), and (2) the index of the input message + wait (e.g., "`in{i},wait`"). Each entry in the INPUTCHECKLIST serves as a key to look up the emulated output O_{emul} in the `output_map`. If no input is found (i.e., the output is caused by a periodic timer), the replicator creates an entry with a "default" key in the INPUTCHECKLIST. Similarly, we store an index key "default" and a default output value in the `output_map` in Algorithm 3. The default value can be obtained from the PLC configuration file. After populating the INPUTCHECKLIST, the replicator assigns a timer to the real PLC output to schedule when to check the emulated PLC outputs. The timer has an expiry time of `detection_delay`, which is defined as replication delay and wait delays at the respective emulators (including the delay for each emulator to process and output a message). We can reduce the delay the replicator has to wait for an output by reducing the periodic timer at the emulator. The periodic timer corresponds to the wait delay in our definition of `detection_delay`. When the timer expires, the INPUTCHECKLIST array will be passed as a parameter to the callback function `msg_matching_callback()` to check the emulated PLC outputs.

Algorithm 5. When the timer with `detection_delay` expires, the comparator will find the valid output O_{emul} from the `out_map` by referencing the keys inside the INPUTCHECKLIST and checking whether at least one O_{emul} value (with matching key in the INPUTCHECKLIST array) matches the real output O_{real} . Otherwise, our logic will raise a modification alarm.

Algorithm 4: Handle the real PLC's output message

Input: `InQueuereal` : Queue stores real PLC's input messages
Output: Generate a timer with `detection_delay` for the callback of `msg_matching_callback`

```

1 Function handle_real_output_msg(Oreal) :
2   current_msg_count = current_msg_count + 1
   // Start a timer to handle timer-triggered
   periodic outputs
3   Timer(periodic_out_msg_callback, Πm,
   current_msg_count).start()
4   inputchecklist = []
5   for each Ireal in reverse(InQueuereal) do
6     if Ireal.index ≤ Oreal.index then
7       tinterval = Oreal.timestamp -
       Ireal.timestamp
8       inputchecklist.append(Ireal.index, wait)
9       inputchecklist.append(Ireal.index)
10      if tinterval > ΔMax then
11        break
12      end
13    end
14  end
   // Start a timer to wait for assigned
   emulators to output messages for comparison
15  Timer(msg_matching_callback, detection_delay, Oreal,
   inputchecklist).start()

```

Algorithm 5: Handle real PLC's output timer timeout

Input: `Oreal` : real PLC's output message.

Output: Raise alerts if any discrepancy is detected

```

1 Function msg_matching_callback(Oreal,
   inputchecklist) :
2   found = false
3   for each key in inputchecklist do
4     Oemul ← output_map(key)
5     if Oreal.packet == Oemul.packet then
6       found = true
7       update_other_emulators_step(key)
8       break
9     end
10  end
11  if found == false then
12    Raise "MODIFICATION ALARM"
13  end

```

C. Dealing with Transient Uncertainty (Burst of Messages)

In Section V-B, we instantiate two PLC emulators to handle one input message. In the event that multiple input messages arrive at the real PLC, the two PLC emulators (i.e., emulated PLC1 and PLC2) may still be busy and unavailable. In this case, subsequent input messages may have to wait longer to be processed, effectively extending beyond `detection_delay`. When that happens, the "newer" input messages may not be processed in time to update `output_map` and the real PLC output message will be treated as invalid, causing false alarms. We address this problem by instantiating multiple emulated PLCs. In other words, we ensure that there is always a set of available PLC emulators to process the "new" input messages.

With reference to Fig. 6, we first explain the handling of one input message using five PLC emulators. At time $\leq t_1$, there is no input message and thus, the status of the five PLC emulators

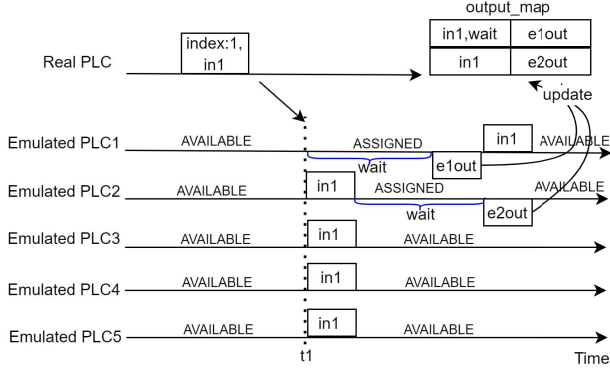


Fig. 6: An example with five emulated PLCs and one input message to illustrate the inner working of DNAttest.

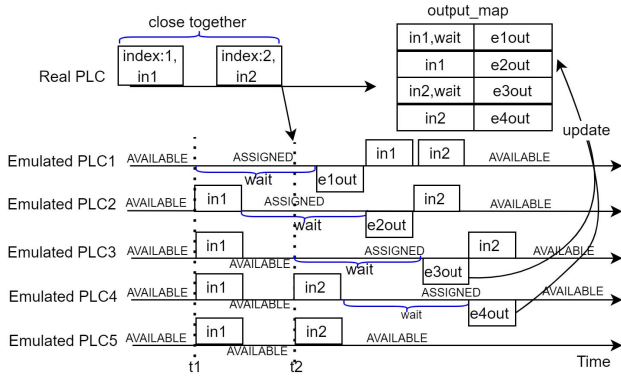


Fig. 7: An example with five emulated PLCs and two input messages to illustrate the inner working of DNAttest.

are AVAILABLE. When input message $in1$ arrives at t_1 , the first two PLC emulators (i.e., emulated PLC1 and emulated PLC2) will be assigned to process $in1$ and their status will change from AVAILABLE to ASSIGNED. Specifically, the emulated PLC1 will wait for the output $e1out$ (caused by a periodic timer) to appear before processing input message $in1$. The emulated PLC2 will process input message $in1$ directly without delay. As for the remaining PLC emulators (i.e., emulated PLC3 to PLC5), we let them process the same input message so that they are in sync with the real PLC state to handle the next input message if available. Although emulated PLC3 to PLC5 handle $in1$, their status remain AVAILABLE.

In Fig. 7, we illustrate the case when another input message $in2$ arrives shortly behind the first input message $in1$ at the real PLC at t_2 . Since the first two PLC emulators (i.e., emulated PLC1 and emulated PLC2) are still processing $in1$ and are in the ASSIGNED status, they cannot handle the second input message $in2$. Therefore, emulated PLC3 and emulated PLC4 will be used to handle $in2$. Note that emulated PLC1 and emulated PLC2 can only start to process $in2$ after they finish processing the first input message $in1$. The emulated PLC3 and emulated PLC4 after receiving $in2$ will change from AVAILABLE to ASSIGNED. In line with

Algorithm 6: Handle the delete, delay, and inject attack for periodic output messages from the PLC

Input: $current_msg_count$: number of received packets
Function $periodic_out_msg_callback()$
1 **Function** $periodic_out_msg_callback()$
2 $recorded_msg_count: integer$:
3 **if** $current_msg_count - recorded_msg_count > 1$ **then**
4 | Raise "INJECTION ALARM";
5 **else**
6 | **if** $current_msg_count == recorded_msg_count$ **then**
7 | | Raise "DELETION or DELAY ALARM";
8 | **end**
9 **end**

Algorithm 2, emulated PLC5 will be given $in2$ but its status will remain AVAILABLE.

The outputs from each of the emulated PLCs are then updated into the output_map. Upon expiry of the real PLC's output timer (i.e., after $detection_delay$), the comparator will check if there is a match in the output_map in Algorithm 5. To optimize the system performance, whenever a real output is found in the output map, it will notify other PLC emulators that have been assigned to process the corresponding input messages immediately so as to bring all those assigned emulated PLCs to the latest state to catch up with real PLC.

D. Dealing with Deletion, Injection and Delay Attacks.

In the previous subsection, we design a mechanism to identify possible output values due to various transient cases. This allows us to deal with modification attack on output messages. To detect deletion, delay and injection attacks on periodic output messages (i.e., they are generated based on some internal timers in the PLC), we start a timer with an expiry time of Π_m in the $handle_real_output_msg()$ function of Algorithm 4 to invoke $periodic_out_msg_callback()$ upon timer expiry in Algorithm 6.

Based on network traces collected on the EPIC testbed, we observed that PLC messages are sent periodically, Π_m defined in Section IV-D. Thus, we define a message counter ($current_msg_count$) to deal with injection, deletion, and delay attacks. When an output message is received at the real PLC, it will update this counter (see Line 2 of Algorithm 4, create a timer with a delay of $output_msg_period$, and pass the message count ($recorded_msg_count$) as an argument to the callback function. When the timer expires, the callback function $periodic_out_msg_callback$ will compare the current message counter and the recorded message counter. If there is no attack, the current message counter should increase by exactly one during this period. If the difference is greater than one, it implies an injection attack within the periodic interval Π_m and our algorithm will raise an injection alarm. If the difference is zero (i.e., the real PLC did not send an output message), it indicates a deletion or delay attack.

VI. CASE STUDY — EPIC TESTBED

We apply our solution to the the Electric Power and Intelligent Control (EPIC) testbed [13], [14] to demonstrate its applicability. The EPIC testbed is a real-world system that mimics realistic power flows from generation, transmission,

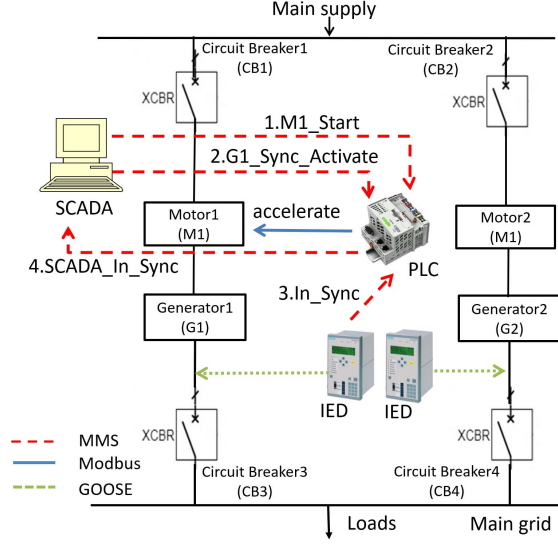


Fig. 8: Synchronization process

to distribution. Each stage has its own PLCs (the testbed uses Wago 750-8202/025-001 PLCs), network switches, power supply units, and protection units such as intelligent electronic devices (IEDs). The IEDs use the IEC 61850 — Generic Object Oriented Substation Event (GOOSE) protocol to communicate real-time system status with each other. The PLC and IEDs communicate with SCADA using IEC 61850 — MMS protocol [41] to receive control commands. In addition, the PLC communicates with the actuators using Modbus protocol to control physical processes.

A. Generator Synchronization Process

We focus on the distribution stage, particularly the PLC that synchronizes the generators to the main grid to supply larger loads. The incoming generator's phase angle must be in sync with the adjacent generator as connecting an out-of-sync generator may cause power instability problems, as reported in [42], [43]. As shown in Fig. 8, each of the generators is controlled by a motor, i.e., M1 or M2. Before synchronizing G1, it is assumed that circuit breakers, CB3 is open, CB1, CB2, and CB4 are closed, and the motor (connected to G2) is rotating at a specified speed. The PLC then executes the control logic (in Listing 1) to synchronize G1 to the grid.

```

1 IF M1_Start = TRUE THEN
2   Modbus.M1_Command := 16#1D4C;
3 END_IF;
4 IF G1_Sync_Activate = TRUE THEN
5   Modbus.M1_Command := 16#1D4E;
6 END_IF;
7 IF In_Sync = TRUE THEN
8   Modbus.M1_Command := 16#1D4C;
9 END_IF;
10 SCADA_In_Sync := In_Sync;

```

Listing 1: Synchronization logic in ST language

Lines 1-3: The SCADA workstation sends an MMS command (M1_Start) to command the PLC to start M1. The PLC responds by sending a Modbus command to rotate M1

TABLE II: PLC outputs under no transient uncertainty

No	Command	Output
1	M1_Start = TRUE	$0 \rightarrow 1D4C_{16}$
2	M1_Start = FALSE	$1D4C_{16}$
3	G1_Sync_Activate = TRUE	$1D4C_{16} \rightarrow 1D4E_{16}$
4	G1_Sync_Activate = FALSE	$1D4E_{16}$

at a speed of $1D4C_{16}$ (or 7500). Lines 4-6: The SCADA sends an MMS command (G1_Sync_Activate) to synchronize G1. Upon receiving the command, the PLC sends a Modbus command to rotate M1 at a higher speed, i.e., $1D4E_{16}$ (or 7502) to catch up with M2 that is rotating at normal speed. The difference in the two motor's rotating speeds will cause the generators' phase angle difference to decrement at a constant rate. This phase angle change is monitored by the IED. The PLC will periodically send MMS commands to the IED to query the phase angle difference. After some time, the phase angle difference will reach near zero, and the IED will close CB3 and respond by sending an MMS command (In_Sync) to the PLC. Lines 7-9: When the PLC receives In_Sync command from the IED, it will send a Modbus command to reset M1 to normal speed of $1D4C_{16}$ (or 7500). The two generators are now synchronized and can operate in parallel to supply power to the grid. Finally, the PLC updates the SCADA by sending an MMS command (SCADA_In_Sync) to complete the synchronization process. Table II shows the different commands and the corresponding outputs of the PLC.

B. Attacks on Generator Synchronization Process

Command Modification (CM) attacks: This attack aims to modify the integrity of the control logic. Critical set points can be manipulated to cause unstable system responses. These modifications can be activated on the basis of some predetermined inputs or a timer. For example, when a synchronization signal is seen, attackers can change the Modbus command to cause the motor to overspin.

Command Injection (CI) attacks: This attack involves injecting malicious code to alter the behavior of actuators. In more severe cases, the attack can render the actuator inoperable. For example, the attacker can inject a malicious Modbus command to instruct the motor overspin and underspin repeatedly, causing permanent damage over time.

Command Deletion/Delay (CD) attacks: The goal is to delete or delay the transmission of commands to actuators to prevent the execution of critical operations. This attack is harder to detect because the payload data is not modified. Attackers can insert timer instructions (e.g., TON timer) into the PLC code to arbitrary delay output signals or simply remove the output commands. This attack has the potential to delay the synchronization process indefinitely.

VII. IMPLEMENTATION

Fig. 9 shows the proposed setup for modeling the generator synchronization process. The real side consists of SCADA, PLC, IED, and motors (i.e., M1 and M2). The PLC acts as a server to respond to MMS requests sent by the SCADA and acts as a client to send MMS and Modbus commands to the IED and motors. All MMS and Modbus packets are forwarded

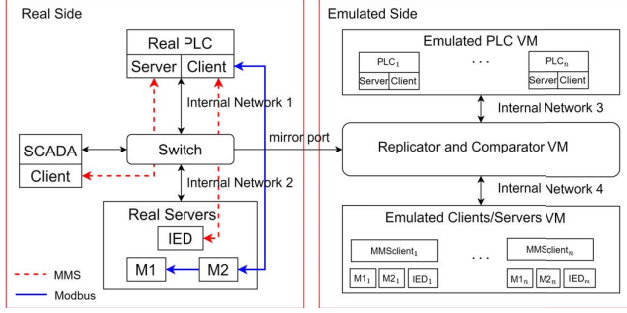


Fig. 9: The DNAttest prototype implementation

TABLE III: Specifications of VMs

Virtual Machine	OS	Processors	RAM
PLC VM	Windows 7	1 core, intel i7-10700	2GB
Replicator VM	Ubuntu 18.04	1 core, intel i7-10700	2GB
Server/client VMs	Ubuntu 18.04	1 core, intel i7-10700	0.5GB

to the emulated part via the mirror port of the switch. The switch is configured to forward packets from the real side while discarding packets from the emulated side.

The emulated part consists of three VMs. The first VM replicates packets to the PLC emulator and implements comparator logic to check the output of the real PLC. The second VM instantiates emulated server and client devices, i.e., virtual IEDs and virtual motors to manage TCP connections with the PLC emulator and the MMS client for packet forwarding. The third VM emulates the PLC control logic. The specifications of each VM are listed in Table III.

Replicator/Comparator VM: Fig. 10 shows the operating states of the replicator. In the initial state, the replicator acts as a bridge to relay TCP packets between the emulated PLC and the emulated servers or between emulated PLC and emulated MMS clients which allows MMS commands from the real SCADA to be forwarded to the emulated PLC. As shown in Fig. 10, there are four TCP connections because the PLC emulator communicates with four different devices (i.e., SCADA, IED, M1, M2). Each device is assigned a different IP address to uniquely identify a TCP connection. As a relay, the replicator keeps track of the latest TCP sequence number, the TCP acknowledgment number, and the IP identification field. If the packet is an MMS packet, the replicator records the invokeID, otherwise, it records the Modbus Transaction Identifier. Once the four TCP connections are successfully established, the replicator will enter the switch-over state to commence packet forwarding. The dispatch module will replicate all received packets by replacing the TCP header fields (i.e., sequence and acknowledgment numbers), IP identification field, including MMS invokeID or Modbus Transaction Identifier before sending them to the corresponding clients and servers in the emulated PLC.

Emulated clients/servers VM: We use Python to develop Modbus TCP servers for M1 and M2. The servers are created to handle TCP requests from the two Modbus clients in the emulated PLC. To emulate the MMS server, we use OpenIEC61850 [44]. We also use Python to create an MMS client

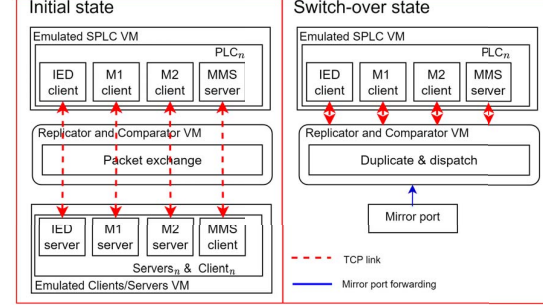


Fig. 10: Initial vs switch-over state for one emulated PLC

to connect to the MMS server in the PLC. All clients/servers can be deployed on the same Linux VM but must be separated by different network namespaces, i.e., each network namespace is assigned a network interface and a unique IP address. The number of emulated client/servers that a VM can support is limited by the number of network interfaces. Since VirtualBox has eight interfaces, each VM can host a maximum of eight emulated setups. Thus, we need 2 VMs to host 12 servers and 4 clients to account for four sets of emulated PLCs. These servers are temporary. Once the initial state of the system is complete, the servers will be released.

Emulated PLC VM: CoDeSys is a development environment for programming PLC applications [30]. It provides a run-time simulator called PLCwinNT, which allows users to test PLC code on a PC or a laptop in simulation mode. We use the windows 7 OS VM to deploy multiple PLCWinNT instances. Each instance runs the same PLC control logic but with different IP and port configurations.

VIII. EVALUATION

Several experiments were conducted to validate the need to deal with transient uncertainty and the effectiveness of DNAttest in attesting the integrity of PLC code in terms of overhead, accuracy, and timeliness of detection.

A. Observe Transient Uncertainty on PLC

The two key design parameters of DNAttest are Δ_{Min} and Δ_{Max} . These parameters define the transient period during which input messages may or may not be processed by a PLC. Specifically, the `handle_real_output_msg` function in Algorithm 3 uses Δ_{Max} to determine how many input messages fall within the transient period, which in turn determines how many PLC emulators to instantiate. In this experiment, we measure Δ_{Min} and Δ_{Max} by sending 10,000 MMS commands to the Wago PLC used in the power system testbed. The MMS command will instruct the PLC to send a Modbus command to rotate the motor from 0 to 7500 (using the synchronization logic as an example). If the first output Modbus packet value is 0 instead of 7500, it means that the input MMS command message has not been processed by the PLC. We record and measure the time interval between each MMS packet and the following first outgoing Modbus packet to determine Δ_{Min} and Δ_{Max} . Fig. 11 shows the time distribution of transient uncertainty and normal intervals.

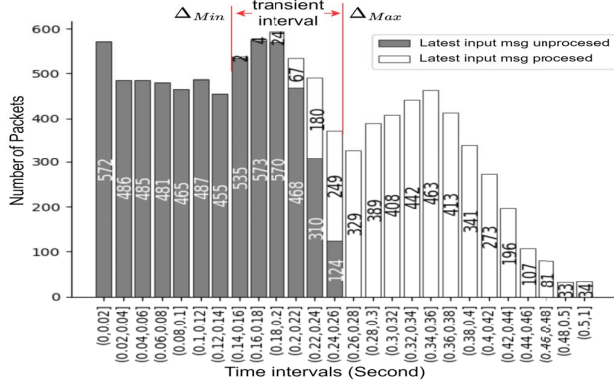


Fig. 11: Transient uncertainty measurement. The time interval is measured between an MMS packet and its immediate outgoing Modbus packet. The transient interval is determined based on checking if there is a change in the Modbus output (0 → 7500). No change indicates that the MMS packet has not been processed by the PLC and denotes transient uncertainty.

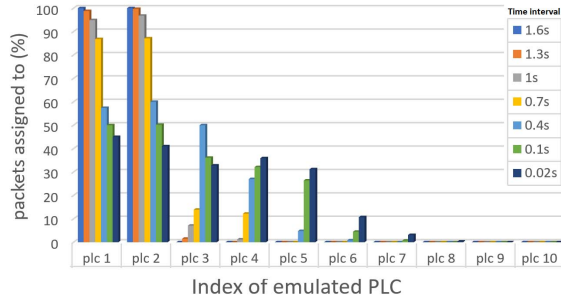


Fig. 12: The usage of emulated PLCs with varying time interval of input messages. Each bar value shows the percentage of input messages that trigger the use of the corresponding emulated PLC for attaining the results.

Our results indicate that transient uncertainty happens between 0.14s (Δ_{Min}) and 0.26s (Δ_{Max}). In practice, we need to calculate Δ_{Max} in a similar way and configure it in DNAttest (with some buffer). For Δ_{min} , it turns to be a small number, and it can be set to 0 for simplicity, without incurring any false alarms (as this allows a larger set of plausible outputs for the real PLC, which are technically possible if the processing delay is 0).

B. Overhead of DNAttest

We evaluate the number of emulated PLCs needed while changing the time interval between two adjacent input messages to a PLC. We setup 10 emulated PLCs and send 1000 MMS messages to the PLC, with the time interval between two adjacent messages set to 1.6s, 1.3s, 1s, 0.7s, 0.4s, 0.1s and 0.02s, in different settings. Figure 12 shows the experiment results. The vertical axis shows the percentage of packets assigned to different instance of the ten emulated PLCs. The horizontal axis shows the emulated PLC 1 to 10. For each PLC, the different bars show the percentages of packets assigned to that PLC under different time intervals. Recall that

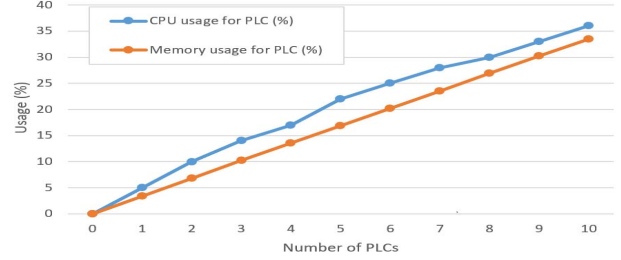


Fig. 13: CPU and memory usage vs. number of emulated PLCs

for each input message, it is assigned to two emulated PLCs. When the time interval is 1.6s, there is enough time for the assigned PLCs to finish processing a given message before the next message arrives, therefore emulated PLC1 and PLC2 process all the 1000 packets. Hence, the bar value is 100% for both emulated PLC1 and PLC2, while 0% for all the others. As the time interval decreases, we observe that the emulated PLC1 and PLC2 are unable to process all the messages in time. Therefore, DNAttest assigns messages to other emulated PLC instances to process the messages. However, even as the time interval reduces to 0.02 second, eight emulated PLCs are sufficient to process all the messages. When a larger number of messages fall in the transient period, the number of emulated PLCs must increase linearly according to the traffic intensity and the duration of the transient interval.

The core processing load of DNAttest is due to the emulated PLC VM. In our design, the VM has one core and 2 GB of RAM. We evaluate the memory and CPU usage performance by simulating different numbers of PLCWinNT on the VM. The results in Figure 13 show that a linear increase in memory and CPU utilization rate as the number of PLCWinNTs increases. Although the usage increases, each PLCWinNT only consumes about 3.5% of memory and 4% of CPU. If we use 10 emulators to monitor every actual PLC, a typical desktop PC with an 8-core Intel i7-10700 CPU and 16 GB of memory (the whole setup costs less than 2000 USD) can attest more than $8 / (0.04 \times 10) = 20$ actual PLCs — which are sufficient to attest all the PLCs in a medium-size ICS.

C. Performance Evaluation

We evaluate DNAttest's detection capabilities against modification, injection, and deletion/delay attacks in Section VI-B by comparing it with two baselines. BASELINE-I implements exact matching of the real PLC's output and the emulator's output. It ignores transient uncertainty of the PLC's behavior. BASELINE-II implements Algorithm 1 to suppress false alarms. One can also use intrusion detection system (IDS) solutions to detect attacks. The difference is that existing IDS solutions do not assume direct access to the PLC's binary images, nor their logic. Instead, they monitor network statistics or use machine learning to learn a model for normal traffic and detect deviations.

As such, we compare DNAttest with a machine learning-based IDS utilizing Long Short Term Memory (LSTM) recurrent neural network, and a statistical-based IDS called

TABLE IV: Detection results across different schemes

Attack	Baseline-I			Baseline-II			DNAttest			LSTM-based IDS			EDMAND		
	Acc.	Prec.	Recall	Acc.	Prec.	Recall	Acc.	Prec.	Recall	Acc.	Prec.	Recall	Acc.	Prec.	Recall
CM	0.921	0.778	1	0.932	0.941	0.837	1	1	1	0.994	0.991	0.997	0.975	1	0.964
CI	0.902	0.715	1	0.927	0.926	0.901	1	1	1	0.930	0.981	0.889	0.916	0.932	0.901
CD	0.896	0.726	1	0.914	0.919	0.861	1	1	1	0.998	0.992	1	0.890	0.862	0.924

EDMAND [45]. Our implementation of the LSTM-based IDS analyzes time-series network traces to build a classifier for detecting suspicious patterns. It is capable of modeling temporal events that characterize the traffic flows corresponding to the plant operations. The EDMAND IDS monitors network traffic at different levels (i.e., transport, control, operation), collect key statistics and use statistical analysis to detect anomalies.

To simplify the experiment, we send only two kind of commands instead of all the commands that appear during the synchronization process. They are G1_sync_Activate = True and G1_sync_Activate = False commands. When G1_sync_Activate = True, the PLC should send a Modbus command to rotate M1 at 7502 (normal operation), otherwise, it is an attack. On the other hand, motor M2 should continue to rotate at 7500. When G1_sync_Activate = False, we set the speed of M1 to zero. For each experiment, we send 1000 sets of G1_Sync_Activate = True and G1_Sync_Activate = False separated by 1 second. We run the three schemes against normal traffic, including the following three types of attacks:

- **Command Modification (CM) attacks:** When the command G1_Sync_Activate = True, the value of Modbus.M1_Command is modified to 7662.
- **Command Injection (CI) attacks:** When G1_Sync_Activate = True, we inject Modbus.M2_Command = 14 command to simulate an injection attack. When G1_Sync_Activate = False, we reset Modbus.M2_Command to zero.
- **Command Delete/Delay (CD) attacks:** We simulate a delete attack by removing Modbus.M1_Command = 7502 command when G1_Sync_Activate = True.

All schemes are evaluated in terms of accuracy ($acc = (TP + TN)/(TP + FP + FN + TN)$), precision ($prec = TP/(TP + FP)$) and recall ($TP/(TP + FN)$), where TP , FP , TN , and FN denote the true positives, false positives, true negatives, and false negatives respectively. Table IV shows that DNAttest achieves 100% accuracy as expected. BASELINE-I has 100% Recall (i.e., zero false negative), but low precision (i.e., very high false positive rate). While BASELINE-II improves the precision, it reduces the recall, which means that it may miss some attacks. Also note that Baseline-II does not account for transient uncertainty, so its precision is not 100%. Although LSTM-based IDS can detect CM and CD attacks with over 99% accuracy (and worse for CI attack), it is inferior to DNAttest. It is because most machine learning-based IDS require high-quality data to train accurate classifiers. If machine learning-based IDS is not trained on unseen attacks, the classifier could easily misclassify between normal and an anomaly, resulting in high false negatives. Similarly, EDMAND's detection capability is

sub-par to DNAttest due to sampling bias and stealthy attacks that inject or modify values close to statistical norms.

Finally, we measure the delay for attesting output messages from the target PLC. We record the timestamp difference when the replicator receives the real side's output and when the assigned emulated PLC instances generate the required output for the comparator. This attestation delay is about 1.1s in our experiment. We can reduce the attestation delay by decreasing the periodic time interval of the emulator, thus making it produce outputs faster and reducing the time for which the replicator must wait before comparing the outputs. We conclude that DNAttest can provide near real-time attack detection capability with around 1 second of delay. In comparison, EDMAND IDS incurs on average 2 seconds of detection delay, while the LSTM-based IDS can incur greater than 200 seconds of delay as it needs to examine a series of timestamped data to make the decision.

IX. CONCLUSION

We designed and implemented a non-invasive digital-twin-based framework for attesting the software integrity of PLCs in an ICS environment. DNAttest is based on emulating the PLC control logic on a virtualized platform and replicating real-world messages to the PLC emulator to generate outputs for cross-checking the actual PLC runtime behavior. In developing DNAttest, we addressed the transient uncertainty of the PLC behavior. Our solution requires no hardware or software changes to existing PLCs and can be applied to legacy ICS networks. We ran extensive tests on the EPIC testbed to demonstrate that our DNAttest can detect various attacks with high accuracy. In the future, we plan to deploy DNAttest on other ICS networks to demonstrate its practicality.

ACKNOWLEDGMENT

This research is supported in part by the National Research Foundation, Singapore, under its National Satellite of Excellence Programme "Design Science and Technology for Secure Critical Infrastructure" (Award Number: NSoE_DeST-SCI2019-0008 and NSoE_DeST-SCI2021TG-0003), in part by National Research Foundation and Infocomm Media Development Authority under its Future Communications Research and Development Programme (FCP-SUTD-RG-2021-016), in part by the SUTD Start-up Research Grant (SRG Award No: SRG ISTD 2020 157), and in part by SUTD-ZJU IDEA Grant for visiting professors (SUTDZJUV201901). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation and Infocomm Media Development Authority, Singapore.

REFERENCES

- [1] D. Pliatsios, P. Sarigiannidis, T. Lagkas, and A. G. Sarigiannidis, "A Survey on SCADA Systems: Secure Protocols, Incidents, Threats and Tactics," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1942–1976, 2020.
- [2] K. E. Hemsley, E. Fisher *et al.*, "History of Industrial Control System Cyber Incidents," Idaho National Lab.(INL), Idaho Falls, ID (United States), Tech. Rep., 2018.
- [3] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [4] A. Di Pinto, Y. Dragoni, and A. Carcano, "TRITON: The First ICS Cyber Attack on Safety Instrument Systems," in *Proc. Black Hat USA*, vol. 2018, 2018, pp. 1–26.
- [5] J. Slowik, "Crashoverride: Reassessing the 2016 Ukraine Electric Power Event as a Protection-focused Attack," *Dragos, Inc.*, 2019.
- [6] H. C. Tan, C. Cheh, B. Chen, and D. Mashima, "Tabulating cybersecurity solutions for substations: Towards pragmatic design and planning," in *2019 IEEE Innovative Smart Grid Technologies-Asia (ISGT Asia)*. IEEE, 2019, pp. 1018–1023.
- [7] L. Garcia, F. Brasser, M. H. Cintuglu, A.-R. Sadeghi, O. A. Mohammed, and S. A. Zonouz, "Hey, My Malware Knows Physics! Attacking PLCs with Physical Model Aware Rootkit," in *NDSS*, 2017.
- [8] N. Govil, A. Agrawal, and N. O. Tippenhauer, "On Ladder Logic Bombs in Industrial Control Systems," in *Computer Security*. Springer, 2017, pp. 110–126.
- [9] Z. Basnight, J. Butts, J. Lopez Jr, and T. Dube, "Analysis of Programmable Logic Controller Firmware for Threat Assessment and Forensic Investigation," *Journal of Information Warfare*, vol. 12, no. 2, pp. 1–9, 2013.
- [10] NIST, "CVE-2022-1161 Detail," 2022. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-1161>
- [11] —, "CVE-2022-1159 Detail," 2022. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-1159>
- [12] S. Bratus, N. D'Cunha, E. Sparks, and S. W. Smith, "TOCTOU, Traps, and Trusted Computing," in *International Conference on Trusted Computing*. Springer, 2008, pp. 14–32.
- [13] A. Siddiqi, N. O. Tippenhauer, D. Mashima, and B. Chen, "On practical threat scenario testing in an electric power ics testbed," in *Proceedings of the 4th ACM Workshop on Cyber-Physical System Security*, 2018, pp. 15–21.
- [14] M. R. Saifuddin, L. Wei, H. C. Tan, and B. Chen, "Coordinated Network Attacks on Microgrid Dispatch Function: An EPIC Case Study," in *Computer Security. ESORICS 2022 International Workshops: CyberICPS 2022, SECPRE 2022, SPOSE 2022, CPS4CIP 2022, CDT&SECOMANE 2022, EIS 2022, and SecAssure 2022, Copenhagen, Denmark, September 26–30, 2022, Revised Selected Papers*. Springer, 2023, pp. 26–45.
- [15] T. Abera, N. Asokan, L. Davi, J.-E. Ekberg, T. Nyman, A. Pavard, A.-R. Sadeghi, and G. Tsudik, "C-FLAT: Control-flow Attestation for Embedded Systems Software," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 743–754.
- [16] D. Felton, "ARM TrustZone Technology," 2019. [Online]. Available: <https://www.trustonic.com/technical-articles/what-is-trustzone/>
- [17] Intel, "Intel SGX," 2015. [Online]. Available: <https://software.intel.com/sgx>
- [18] Q. Ye, H. C. Tan, D. Mashima, B. Chen, and Z. Kalbarczyk, "Position Paper: On Using Trusted Execution Environment to Secure COTS Devices for Accessing Industrial Control Systems," 2021.
- [19] S. F. J. J. Ankergård, E. Dushku, and N. Dragoni, "State-of-the-Art Software-Based Remote Attestation: Opportunities and Open Issues for Internet of Things," *Sensors*, vol. 21, no. 5, p. 1598, 2021.
- [20] B. Chen, X. Dong, G. Bai, S. Jauhar, and Y. Cheng, "Secure and Efficient Software-based Attestation for Industrial Control Devices with ARM Processors," in *Proceedings of the 33rd Annual Computer Security Applications Conference*, 2017, pp. 425–436.
- [21] R. V. Steiner and E. Lupu, "Towards More Practical Software-based Attestation," *Computer Networks*, vol. 149, pp. 43–55, 2019.
- [22] C. Castelluccia, A. Francillon, D. Perito, and C. Soriente, "On the Difficulty of Software-based Attestation of Embedded Devices," in *Proceedings of the 16th ACM conference on Computer and communications security*, 2009, pp. 400–409.
- [23] Y. Chen, C. M. Poskitt, and J. Sun, "Learning from Mutants: Using Code Mutation to Learn and Monitor Invariants of a Cyber-physical System," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 648–660.
- [24] —, "Code Integrity Attestation for PLCs Using Black Box Neural Network Predictions," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 32–44.
- [25] M. Zhang, C.-Y. Chen, B.-C. Kao, Y. Qamsane, Y. Shao, Y. Lin, E. Shi, S. Mohan, K. Barton, J. Moyne *et al.*, "Towards Automated Safety Vetting of PLC Code in Real-world Plants," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 522–538.
- [26] S. Adepu, F. Brasser, L. Garcia, M. Rodler, L. Davi, A.-R. Sadeghi, and S. Zonouz, "Control Behavior Integrity for Distributed Cyber-physical Systems," in *2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPs)*. IEEE, 2020, pp. 30–40.
- [27] H. C. Tan, C. Cheh, and B. Chen, "CoToRu: Automatic Generation of Network Intrusion Detection Rules from Code," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 720–729.
- [28] F. B. Schneider, "Implementing fault-tolerant services using the state machine approach: A tutorial," *ACM Computing Surveys (CSUR)*, vol. 22, no. 4, pp. 299–319, 1990.
- [29] L. Lamport, R. Shostak, and M. Pease, "The byzantine generals problem," in *Concurrency: the works of leslie lamport*, 2019, pp. 203–226.
- [30] S.-S. S. S. GmbH, "CODESYS," 2018. [Online]. Available: <https://www.codesys.com>
- [31] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-physical System Security: Device Physics Matters too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.
- [32] C. M. Ahmed, A. P. Mathur, and M. Ochoa, "NoiSense Print: Detecting Data Integrity Attacks on Sensor Measurements Using Hardware-based Fingerprints," *ACM Trans. Priv. Secur.*, vol. 24, no. 1, Sep. 2020. [Online]. Available: <https://doi.org/10.1145/3410447>
- [33] J. Giraldo, D. Urbina, C. Tang, and A. A. Cardenas, "The More The Merrier: Adding Hidden Measurements to Secure Industrial Control Systems," in *Proceedings of the 7th Symposium on Hot Topics in the Science of Security*, 2020, pp. 1–10.
- [34] V. Y. Pillitteri and T. L. Brewer, "Guidelines for Smart Grid Cybersecurity," *NIST Pubs*, 2014.
- [35] K. Stouffer, J. Falco, K. Scarfone *et al.*, "Guide to Industrial Control Systems (ICS) Security," *NIST special publication*, vol. 800, no. 82, pp. 16–16, 2011.
- [36] "IEC 61131-3 - Programmable Controllers - ALL PARTS," Jan 2020. [Online]. Available: <https://webstore.iec.ch/publication/62427>
- [37] "Purifyplus," 2022. [Online]. Available: <https://www.teamblue.uniconsi.com/products/purifyplus/>
- [38] "Memory Validator," 2022. [Online]. Available: <https://www.softwareverify.com/product/memory-validator/>
- [39] Y. Z. Gregory J. Duck and R. H. C. Yap, "Hardening Binaries against More Memory Errors," in *EuroSys '22, April 5–8, 2022, RENNES, France*, 2022.
- [40] M. Organization, *MODBUS Messaging on TCP/IP Implementation Guide: V1. 0b*. Modbus Organization, 2006.
- [41] "IEC 61850-8-1:2011+AMD1:2020 CSV Consolidated Version Communication Networks and Systems for Power Utility Automation - Part 8-1: Specific Communication Service Mapping (SCSM) - Mappings to MMS (ISO 9506-1 and ISO 9506-2) and to ISO/IEC 8802-3," Feb 2020. [Online]. Available: <https://webstore.iec.ch/publication/66585>
- [42] N. K. Kandasamy, "An Investigation on Feasibility and Security for Cyberattacks on Generator Synchronization Process," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 9, pp. 5825–5834, 2019.
- [43] M. Zeller, "Common Questions and Answers Addressing the Aurora Vulnerability," *Schweitzer Engineering Laboratories Report*, 2011.
- [44] OpenMUC, "OpenIEC61850 Overview," 2019. [Online]. Available: <https://www.openmuc.org/iec-61850/>
- [45] T. Y. Wenyu Ren and K. Nahrstedt, "EDMAND: Edge-Based Multi-Level Anomaly Detection for SCADA Networks," in *2018 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2018.